CLOUD NATIVE ECOSYSTEM / KUBERNETES

Облачная среда: уровень определения и разработки приложений

Как следует из названия, уровень определения и разработки приложений ориентирован на инструменты, которые позволяют инженерам создавать приложения.

26 января 2021 года, 6:00 Кэтрин Паганини и Джейсон Морган

Этот пост является частью серии публикаций сопредседателей подкомитета по бизнес-ценности Cloud Native Computing Foundation Кэтрин Паганини и Джейсона Моргана, посвященных объяснению каждой категории облачных технологий для нетехнической аудитории, а также для инженеров, которые только начинают работать с облачными технологиями.

Мы добрались до верхнего уровня Cloud Native Computing Foundation облачной среды (если вы пропустили наши предыдущие статьи, то мы рассмотрели введение, а также уровень предоставления ресурсов, уровень выполнения и уровень оркестрации и управления в отдельных статьях). Как следует из названия, уровень определения и разработки приложений редоточен на инструментах, которые позволяют инженерам создавать приложения. Все, о чем мы говорили в предыдущих статьях, было связано с

процесс? Теперь давайте рассмотрим этот последний слой.

Побочное примечание

При рассмотрении облачной среды вы заметите несколько отличий:

- Проекты в больших блоках — это проекты с открытым исходным кодом, размещенные на платформе CNCF. Некоторые из них все еще находятся на стадии инкубации (светло-голубая/фиолетовая рамка), другие уже завершены (темно-синяя рамка).
- Проекты в маленьких белых блоках — это проекты с открытым исходным кодом.
- Продукты в серых блоках являются проприетарными.

Пожалуйста, обратите внимание, что даже во время написания этой статьи мы видели, как новые проекты становились частью CNCF, так что всегда ориентируйтесь на текущую ситуацию — все меняется очень быстро!

База данных

другие приложения могут эффективно хранить и извлекать данные.

Это гарантирует сохранность данных, доступ к ним только для авторизованных пользователей и возможность их извлечения с помощью специальных запросов. Несмотря на то, что существует множество различных типов баз данных с разными подходами, все они в конечном счете преследуют одни и те же цели.

Проблема, которую она решает

Большинству приложений нужен эффективный способ хранения и извлечения данных, обеспечивающий их безопасность. Базы данных делают это структурированно, с использованием проверенных технологий (для этого требуется немало усилий).

TRENDING STORIES

1. Introduction to Cloud Native Computing
2. Microsoft wants to make service mesh invisible
3. Local or Cloud: Choosing the Right Dev Environment
4. Microservices 101
5. Tetrate launches open source marketplace to simplify Envoy adoption

How It Helps

Databases provide a common interface for storing and retrieving data for applications. Developers use these standard interfaces and, in most cases a simple query language, to store, query, and retrieve information in a database. At the same time, databases allow users to continuously back up and save data as well as encrypt and regulate access to it.

Technical 101

We've established that a database management system is an application that stores and retrieves data. It does so using a common language and interface that be easily used by a number of different languages and frameworks.

uses should be driven by its needs and constraints.

With the rise of Kubernetes and its ability to support stateful applications, we've seen a new generation of databases that leverage containerization. These new cloud native databases aim to bring the scaling and availability benefits of Kubernetes to databases. Tools like YugaByte and Couchbase are examples of cloud native databases, although more traditional databases like MySQL and Postgres run successfully and effectively in Kubernetes clusters.

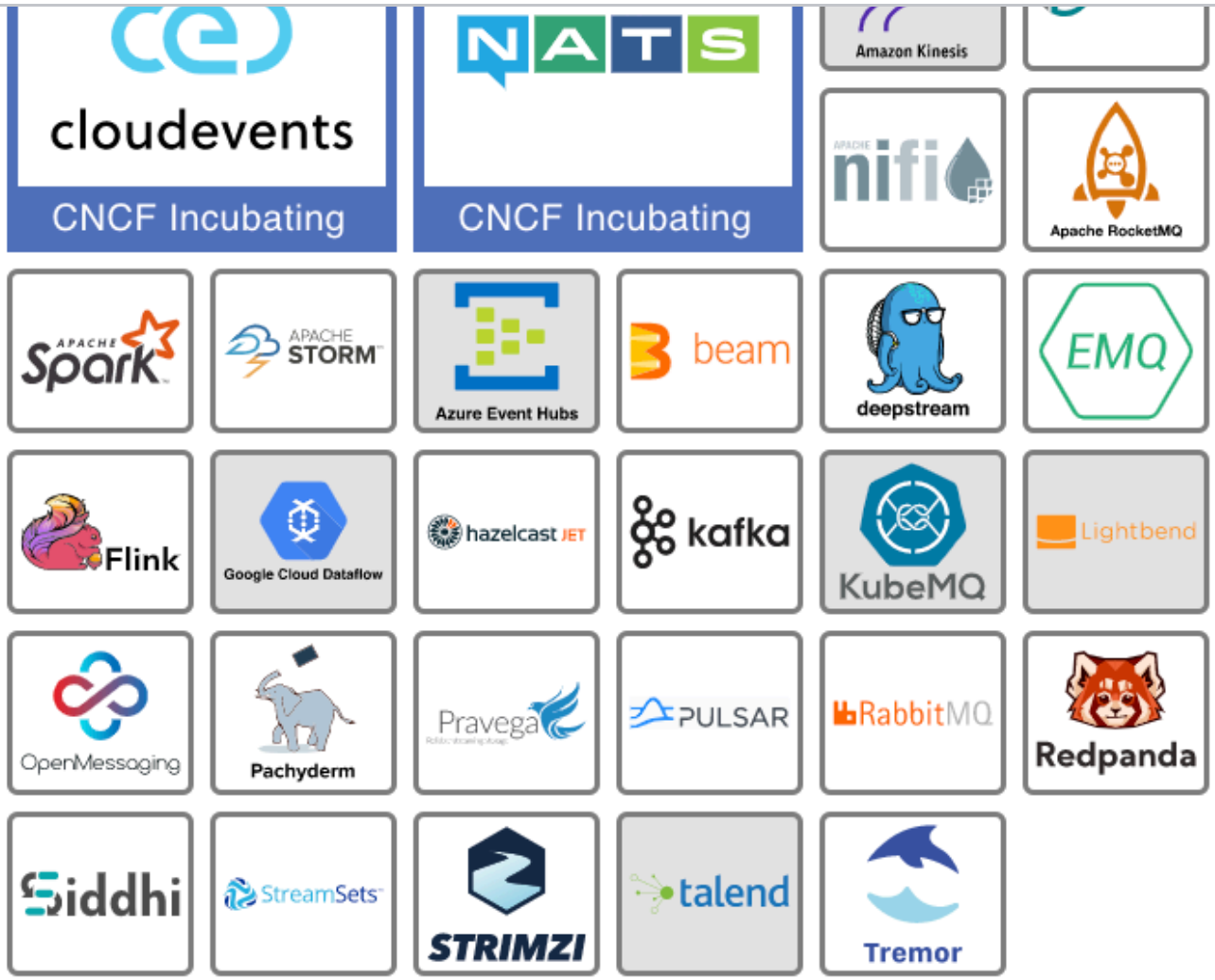
Vitess and TiKV are CNCF projects in this space.

Sidenote

If you look at this category, you'll notice multiple names ending in DB (e.g. MongoDB, CockroachDB, FaunaDB) which, as you may guess, stands for database. You'll also see various names ending in SQL (e.g. MySQL or memSQL) — they are still relevant. Some are "old school" databases that have been adapted to a cloud native reality. There are also some databases that are no-SQL but SQL compatible, such as YugaByte and Vitess.

Buzzwords	Popular Projects
• SQL • DB • Persistence	• Postgres • MySQL • Redis

Streaming and Messaging



Zoom

What It Is

Streaming and messaging tools enable service-to-service communication by transporting messages (i.e. events) between systems. Individual services connect to the messaging service to either publish events, read messages from other services, or both. This dynamic creates an environment where individual apps are either publishers, meaning they write events, or subscribers that read events, or more likely both.

Problem It Addresses

As services proliferate, application environments become increasingly complex, making the orchestration of communication between apps more challenging. A streaming or messaging platform provides a central place to publish and read all the events that occur within a system, allowing applications to work together without necessarily knowing anything about one another.

...ow It Helps

types of events subscribe and watch the streaming or messaging tool. That's the essence of a publish-subscribe, or just pub-sub, approach and is enabled by these tools.

By introducing a "go-between" layer that manages all communication, we are decoupling services from one another. They simply watch for events, take action, and publish a new one.

Here's an example. When you first sign up for Netflix, the signup service publishes a "new signup event" to a messaging platform with further details (e.g. name, email address, subscription level, etc.). The account creator service, which subscribes to signup events, will see the event and create your account. A customer communication service that also subscribes to new signup events will add your email address to the customer mailing list and generate a welcome email, and so on.

This allows for a highly decoupled architecture where services can collaborate without needing to know about one another. This decoupling enables engineers to add new functionality without updating downstream apps (consumers) or sending a bunch of queries. The more decoupled a system is, the more flexible and amenable to change. And that is exactly what engineers strive for in a system.

Technical 101

Messaging and streaming tools have been around long before cloud native became a thing. To centrally manage business-critical events, organizations have built large enterprise service buses. But when we talk about messaging and streaming in a cloud native context, we're generally referring to tools like NATS, RabbitMQ, Kafka, or cloud provided message queues.

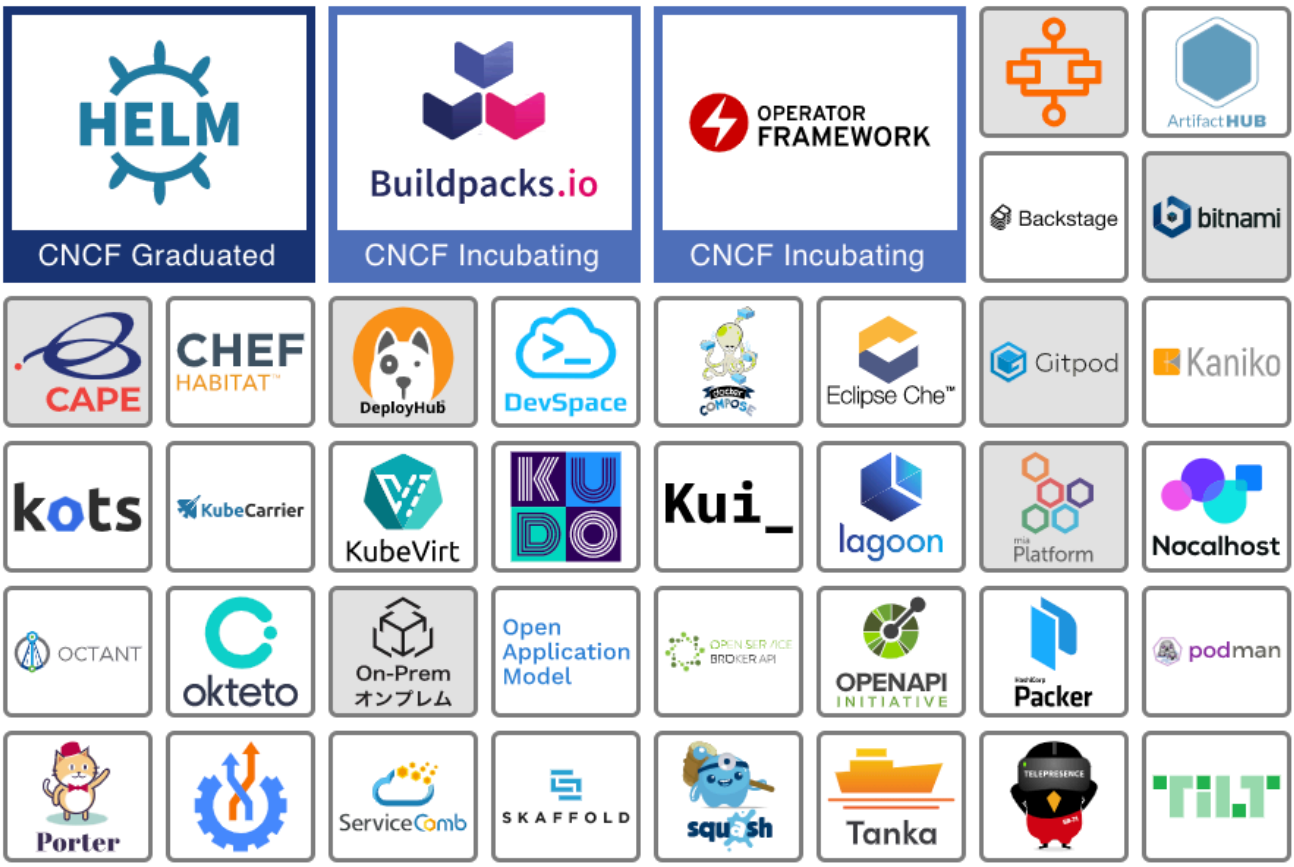
What these systems have in common, is the architectural patterns they enable. Application interactions in a cloud native environment are either orchestrated or choreographed. You can find a lot more details about the topic in Sam Newton's book Building Microservices, as well as brief summaries in this StackOverflow post and this great Medium blog by Chen Chen. To simplify things, let's just say that orchestrated refers to systems that are centrally managed and choreographed are those that allow individual components to act independently.

apps can go to, to both tell others what they're doing by publishing messages, or see what things are going on by subscribing to messages.

The NATS and Cloudevents projects are both incubating projects in this space with NATS providing a mature messaging system and Cloudevents being an effort to standardize message formats between systems. Strimzi, Pravega, and Tremor are sandbox projects with each being tailored to a unique use case around streaming and messaging.

Buzzwords	Popular Projects
• Choreography • Steaming • MQ • Message Bus	• Spark • Kafka • RabbitMQ • Nats

Application Definition and Image Build



Zoom

at It Is

containers and/or Kubernetes. And second, ops-focused tools that deploy apps in a standardized way. Whether to speed up or simplify your development environment, provide a standardized way to deploy third-party apps, or simplify the process of writing a new Kubernetes extension, this category serves as a catch-all for a number of projects and products that optimize the Kubernetes developer and operator experience.

Problem It Addresses

Kubernetes, and containerized environments more generally, are incredibly flexible and powerful. With that flexibility also comes complexity, mainly in form of numerous configuration options for various often new use cases. Developers must containerize their code and have the ability to develop in production-like environments. And with a rapid release schedule, operators need a standardized way to deploy apps into container environments.

How It Helps

Tools in this space aim at solving some of these developer or operator challenges. On the developer side, there are tools that simplify the process of extending Kubernetes to build, deploy, and connect applications. A number of projects and products help to store or deploy pre-packaged apps. These allow operators to quickly deploy a streaming service like Kafka or install a service mesh like Linkerd.

Developing cloud native applications brings a whole new set of challenges calling for a large set of diverse tools to simplify application build and deployments. As you start addressing operational and developer concerns in your environment, look for tools in this category.

Technical 101

App definition and build tools encompass a huge range of functionality. From extending Kubernetes to virtual machines with KubeVirt, to speeding app development by allowing you to port your development environment into Kubernetes with tools like Telepresence, and everything in between. At a high level, tools in this space solve either developer-focused concerns, like how to correctly write, package, test, or run custom apps, or operations-focused concerns, such as deploying and managing applications.

Helm, the only graduated project in this category, underpins many app deployment patterns. Allowing Kubernetes users to deploy and customize many popular third-

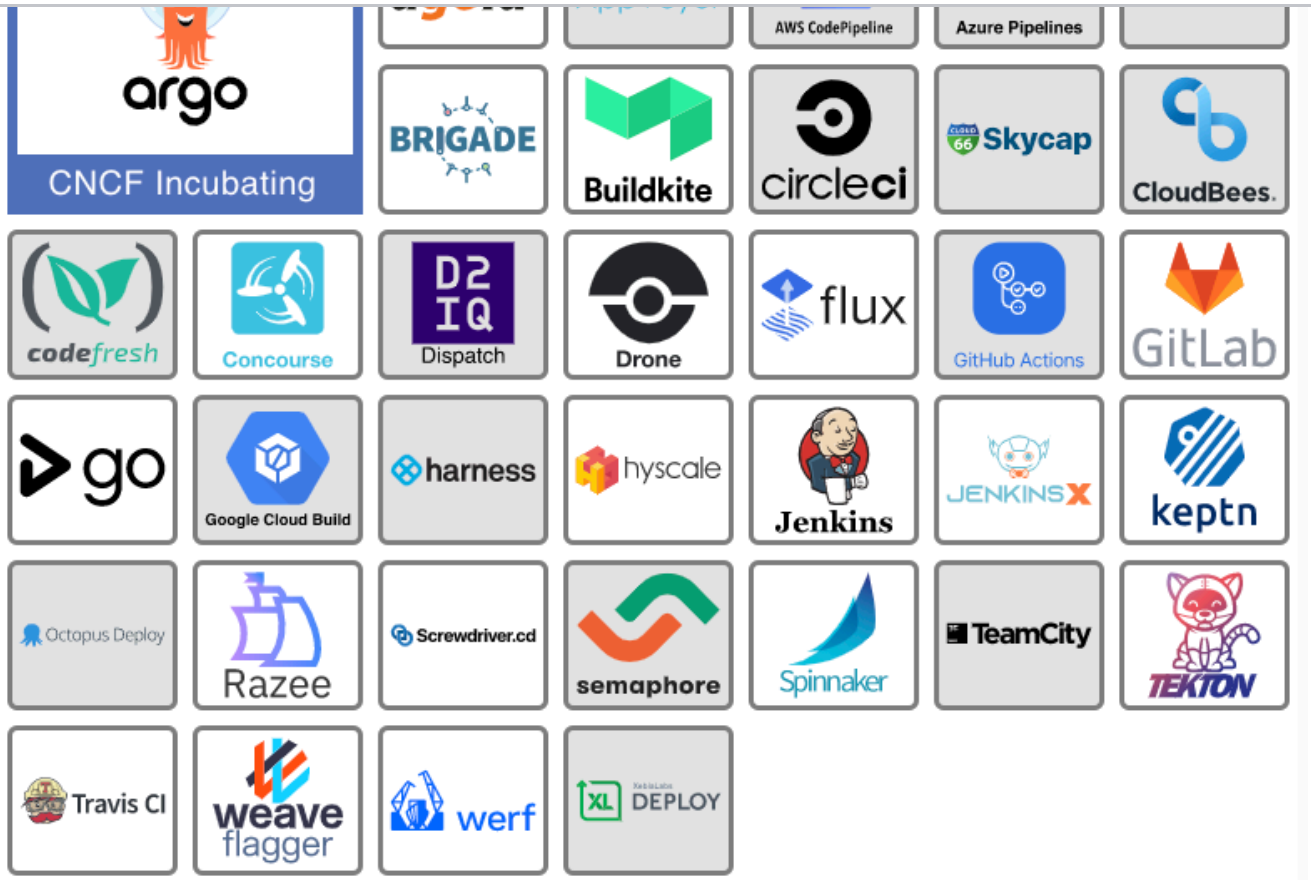
flexible enough to allow users to customize their own app deployments.

The Operator Framework is an incubating project aimed at simplifying the process of building and deploying operators. Operators are out of scope for this article but let's note here that they help deploy and manage apps, similar to Helm (you can read more about operators here). Cloud Native Buildpacks, another incubating project, aims to simplify the process of building application code into containers.

There's a lot more in this space and exploring it all would require a dedicated article. But research these tools further if you want to make Kubernetes easier for developers and operators. You'll likely find something that meets your needs.

Buzzwords	Popular Projects
• Package Management • Charts • Operators	• Helm • Buildpacks • Tilt • Okteto

Continuous Integration and Continuous Delivery



Zoom

What It Is

Continuous integration (CI) and continuous delivery (CD) tools enable fast and efficient development with embedded quality assurance (we cover CI/CD in detail in this primer). CI automates code changes by immediately building and testing the code, ensuring it produces a deployable artifact. CD goes one step further and pushes the artifact through the deployment phases.

Mature CI/CD systems watch source code for changes, automatically build and test the code, then begin moving it from development to production where it has to pass a variety of tests or validation to determine if the process should continue or fail. Tools in this category enable such an approach.

Problem It Addresses

Building and deploying applications is a difficult and error-prone process. Particularly, when it involves a lot of human intervention and manual steps. The longer a developer works on a piece of software without integrating it into the codebase, the longer it will take to identify an error and the more difficult it is to fix. Integrating code on a regular basis, errors are caught early and are easier to

While tools like Kubernetes offer great flexibility for running and managing apps, they also create new challenges and opportunities for CI/CD tooling. Cloud native CI/CD systems are able to leverage Kubernetes itself to build, run, and manage the CI/CD process, often referred to as pipelines. Kubernetes also provides information about the health of our apps enabling cloud native CI/CD tools to more easily determine if a given change was successful or needs to be rolled back.

How It Helps

CI tools ensure that any code change or updates developers introduce are built, validated, and integrated with other changes automatically and continuously. Each time a developer adds an update, automated testing is triggered ensuring only good code makes it into the system. CD extends CI to include pushing the result of the CI process into production-like and production environments.

Let's say a developer changes the code for a web app. The CI system sees the code change, and then builds and tests a new version of that web app. The CD system takes that new version and deploys it into a dev, test, pre-production, and finally production environment. It does that while testing the deployed app after each step in the process. All together these systems represent a CI/CD pipeline for that web app.

Technical 101

Over time, a number of tools have been built to help with the process of moving code from a repository, where the code is stored, to production, where the finished app runs. Like most other areas of computing, the advent of cloud native development has changed CI/CD systems. Some traditional tools like Jenkins, probably the most widely-used CI tool on the market, has been overhauled entirely to better fit into the Kubernetes ecosystem. Others, like Flux and Argo have pioneered a new way of doing continuous delivery called GitOps.

In general, you'll find projects and products in this space are either (1) CI systems, (2) CD systems, (3) tools that help the CD system decide if the code is ready to be pushed into production, or (4), in the case of Spinnaker and Argo, all three. Argo and Brigade are the only CNCF projects in this space but you can find many more options hosted by the Continuous Delivery Foundation. Look for tools in this space to help your organization automate your path to production.

• Continuous integration • Continuous delivery • Blue/Green • Canary Deploy	• Flagger • Spinnaker • Jenkins

Conclusion

As we've seen, tools in the application definition and development layer enable engineers to build cloud native apps. You'll find databases to store and retrieve data or streaming and messaging tools allowing for decoupled, choreographed architectures. App definition and image build tools encompass a variety of technologies that improve the developer and operator experience, and CI/CD ensures code is at a deployable state and helps engineers catch any errors early on, ensuring better code quality.

This article concludes the layers of the CNCF landscape. In our next article, we'll focus on cloud native platforms, the first column running across all these layers. Configuring tools across the layers we discussed so far so they work well together is no easy task. Platforms bundle them together easing adoption, but more to that in our next article.

As always, a very special thanks to Ihor Dvoretskyi from the CNCF who was so kind as to review the article making sure it's all accurate.

The Cloud Native Computing Foundation is a sponsor of The New Stack.

TNS

Catherine Paganini aims to drive diversity and inclusion across the cloud native landscape. As CNCF TAG Contributor Strategy Co-chair and Deaf & Hard of Hearing WG facilitator, she founded and/or facilitates multiple inclusion initiatives (BIPOC, blind/visually impaired, LGBTQ+, stuttering/speech). Her...

Read more from Catherine Paganini →

Jason Morgan is co-chair of the Cloud Native Computing Foundation's Business Value Subcommittee and Developer Evangelist for Linkerd at Buoyant where he helps educate

Read more from Jason Morgan →

SHARE THIS STORY

TRENDING STORIES

1. Introduction to Cloud Native Computing
2. Microsoft wants to make service mesh invisible
3. Local or Cloud: Choosing the Right Dev Environment
4. Microservices 101
5. Tetrate launches open source marketplace to simplify Envoy adoption

SUBSCRIBE

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS